Ingegneria del Software — SOLUZIONE DEL TEMA 10 settembre 2020

Esercizio 1

Si consideri la seguente classe `ImmList<T>` che definisce una lista immutabile di elementi di tipo `T`.

```
public /*@ pure @*/ class ImmList<T> {
  // Ritorna true se e solo se la lista e' vuota
  public boolean empty();

  // Ritorna il primo elemento della lista
  // Lancia EmptyException se la lista e' vuota
  public T first() throws EmptyException;

  // Ritorna una nuova lista identica a this ma senza il primo elemento
  // In caso di lista vuota, ritorna una nuova lista vuota
  public ImmList<T> tail();

  // Ritorna una nuova lista che contiene el in ultima posizione
  // Lancia NullException se el e' null
  public ImmList<T> add(T el) throws NullException;

  // Ritorna il numero di occorrenze dell'elemento el nella lista
  public int count(T el)
}
```

Domanda a)

Si scriva la specifica JML del metodo `add`.

Soluzione

```
//@ensures el != null && \result != null &&
//@this.empty() ==>
//@    (\result.first().equals(el) && \result.tail().empty()) &&
//@!this.empty() ==>
//@    (\result.first().equals(first()) && \result.tail().equals(tail().add(el)))
//@
//@signals NullException el == null
public ImmList<T> add(T el) throws NullException;
```

Domanda b)

Si scriva la specifica JML del metodo `count`.

Soluzione

```
//@ensures
//@empty() ==> \result == 0 &&
//@!empty() ==>
//@    \result == tail().count(el) + first().equals(el) ? 1 : 0
public int count(T el);
```

Domanda c)

Si consideri una classe `ImmList2<T>` che aggiunge un metodo remove che restituisce una nuova lista identica a this ma senza l'ultimo elemento (in caso di lista vuota, restituisce una nuova lista vuota). È possibile definire `ImmList2<T>` come derivata da `ImmList<T>`, rispettando il principio di sostituzione?

Soluzione

Si, in quanto `ImmList2<T>` sarebbe un'estensione pura di `ImmList<T>`, che aggiunge un metodo senza modificare le specifiche degli altri, e non modifica alcuna proprietà della classe `ImmList<T>`.

Domanda d)

Si consideri una classe `ImmList3<T>` in cui il metodo `add` non lancia eccezioni ma permette l'inserimento di elementi nulli. È possibile definire `ImmList3<T>` come derivata da `ImmList<T>`, rispettando il principio di sostituzione?

Soluzione

No, per due ragioni. (1) Si violerebbe la regola dei metodi, dato che il mancato lancio dell'eccezione *non* rafforza la post-condizione del metodo `add`. (2) Si violerebbe la regola delle properità, invalidando la proprietà che la lista non possa contentere valori nulli.

Esercizio 2

Si consideri la seguente classe Java:

```
public class Bank {
  private List<String> accName;
  private List<Double> accBalance;
  public Bank() {
    accName = new ArrayList<String>();
    accBalance = new ArrayList<Double>();
  }
  public void addAccount(String clientName) throws DuplicateAccountException {
    synchronized(accName) {
      synchronized(accBalance) {
        if(accName.contains(clientName)) throw new DuplicateAccountException();
        accName.add(clientName);
        accBalance.add(0.0);
      }
    }
  }
  public double getBalance(String clientName) throws MissingAccountException {
    synchronized(accName) {
      synchronized(accBalance) {
        int pos = accName.indexOf(clientName);
        if(pos==-1) throw new MissingAccountException();
        return accBalance.get(pos);
      }
    }
  }
  public void deposit(String clientName, double amount) throws MissingAccountException {
    synchronized(accName) {
      synchronized(accBalance) {
        int pos = accName.indexOf(clientName);
        if(pos==-1) throw new MissingAccountException();
        double newBalance = accBalance.get(pos) + amount;
        accBalance.set(pos, newBalance);
      }
    }
  }
  public void withdraw(String clientName, double amount) throws MissingAccountException {
    synchronized(accName) {
      synchronized(accBalance) {
        int pos = accName.indexOf(clientName);
        if(pos==-1) throw new MissingAccountException();
        double newBalance = accBalance.get(pos) - amount;
        accBalance.set(pos, newBalance);
      }
    }
} }
```

Domanda a)

La classe è correttamente sincronizzata (ovvero non si verificano né deadlock né potenziali corse critiche)? Motivare la propria risposta.

Soluzione

La sincronizzazione è corretta (seppur irrituale). La doppia sincronizzazione annidata sui due attributi protegge correttamente l'accesso ai dati condivisi e non da luogo a deadlock perché ogni thread acquisisce i due lock nel medesimo ordine.

Domanda b)

Si modifichi il codice dei soli metodi `addAccount` e `getBalance` affinché il metodo `getBalance`, invece di sollevare una eccezione, sospenda il chiamante se il cliente non è presente nella banca, in attesa che il metodo `addAccount` aggiunga l'account. Si suggerisce di prestare particolare attenzione nel modificare i due metodi al fine di evitare di introdurre potenziali deadlock.

Soluzione

```
public void addAccount(String clientName) throws DuplicateAccountException {
  synchronized(accName) {
    synchronized(accBalance) {
      if(accName.contains(clientName)) throw new DuplicateAccountException();
      accName.add(clientName);
      accBalance.add(0.0);
    }
    accName.notifyAll();
  }
}

public double getBalance(String clientName) {
  synchronized(accName) {
    while(!accName.contains(clientName)){
      try { accName.wait(); }
      catch(InterruptedException ex) {ex.printStackTrace();}
      synchronized(accBalance) {
        int pos = accName.indexOf(clientName);
        return accBalance.get(pos);
      }
    }
  }
}
```

Si noti come la chiamata al metodo `wait` in `getBalance` sia fatta prima di acquisire il secondo lock, al fine di evitare potenziali deadlock.

Esercizio 3

Si consideri il seguente frammento di un programma sequenziale, dove y, z sono variabili intere:

```
1  int x=0;
2  int k=10;
3  while (x <= 10 && z>0) {
4      if (z <= y && k>=x)
5          y = y - z;
6      k--;
7      if (y > 0)  x++;
8      else break;
9  }
```

Domanda a)

Si definisca il minimo un numero di casi di test (per y, z) necessari e sufficienti per coprire tutte le istruzioni.

Soluzione

Ne basta uno: $y = 2, z = 1$.

Domanda b)

Si calcoli la condizione logica (path condition) che impone che il ciclo venga eseguito tre volte, uscendo alla terza iterazione eseguendo l'istruzione break.

Soluzione

$z > 0 \wedge y = 3z$

Domanda c)

Si definisca il minimo numero di casi di test necessari e sufficienti per coprire tutte le condizioni e le diramazioni (edge and condition coverage), se ciò è possibile.

Soluzione

Bastano tre casi. Il primo caso consiste in un valore negativo o nullo per z, p.es. $z = 0$.

Il secondo caso è quello già visto per la copertura delle istruzioni (copre il valore vero e falso di $z <= y$ e di $y > 0$).

Il terzo caso di test deve rendere falsi $x <= 10$ e $k >= x$: a tale scopo, occorre eseguire il ciclo while per 11 volte con $y > 0$ in modo da eseguire ogni volta $x++$: basta scegliere $y > 6z > 0$, p. es. $y = 7, z = 1$. In questo modo al termine della sesta iterazione $k == 4, x == 5, y == 1$, quindi $k >= x$ diventa falso: l'assegnamento di y alla riga 5 non può essere eseguito, quindi y resta positivo: dopo altre 5 iterazioni $x == 11$.